Technical Blocker Mitigation — Defender for Office 365

Implementation Solution Guide

Version 1.0.0 · 2026-07-11

Contents

Executive Summary — Technical Blocker Mitigation: Defender for Office 365 5
What you get . . . 5
What this delivers . . . 5
The standard deployment model . . . 5
Boundary . . . 5
Effort . . . 6
1. Solution Overview — Technical Blocker Mitigation: Defender for Office 365 6
What this solution is . . . 6
How Defender for Office 365 evaluation actually works (so you can find the break) . . . 6
The standard deployment model (applied to a blocker) . . . 7
Scope . . . 7
Out of scope . . . 7
Connections and authentication used . . . 7
2. Requirements Guide — Technical Blocker Mitigation: Defender for Office 365 8
2.1 Read-only access to Exchange Online and the Defender portal . . . 8
2.2 Exchange Online Management module available . . . 8
2.3 The affected recipient subset defined . . . 8
2.4 Knowledge of the mail-routing topology . . . 9
2.5 A safe test message method . . . 9
2.6 A place to store the diagnostic output and support bundle . . . 9
Checklist . . . 9
Global Requirements — Defender (category-level) 9
G1. Microsoft 365 / Entra tenant . . . 10
G2. Defender licensing . . . 10
G3. Administrative roles (least privilege) . . . 10
G4. Onboarding reach to endpoints / identity / workloads . . . 10
G5. Network access to Defender service endpoints . . . 10
G6. A pilot ring . . . 10
G7. Log Analytics workspace (where required) . . . 11
G8. Response governance . . . 11
3. Implementation Guide — Technical Blocker Mitigation: Defender for Office 365 11
Naming (set once, reuse verbatim) . . . 11
Step 1 — Assess: pinpoint the blocker (read-only) . . . 11
Step 2 — Assess: analyze mail routing and precedence (read-only) . . . 12
Step 3 — Configure: apply the one targeted fix (manual, documented) . . . 12

Step 4 — Pilot: re-test the fix on the affected recipients only . . . 12
Step 5 — Validate: confirm the policy now applies . . . 12
Step 6 — Expand: resume the stalled rollout . . . 13
Step 7 — Escalate (only if it is a service issue, not your config) . . . 13
5. Testing Guide — Technical Blocker Mitigation: Defender for Office 365 13
Test 1 — Blocker pinpointed (baseline captured) . . . 13
Test 2 — Mail routing reaches EOP . . . 14
Test 3 — Correct policy wins on precedence . . . 14
Test 4 — Safe Links rewriting . . . 14
Test 5 — Safe Attachments detonation . . . 14
Test 6 — Licensing present . . . 14
Test 7 — Before/after evidence produced . . . 14
Test 8 — Escalation bundle valid (only if escalating) . . . 14
6. Troubleshooting Guide — Technical Blocker Mitigation: Defender for Office 365 15
Mail bypasses filtering (routing conflict) . . . 15
Policy configured but never applies (precedence) . . . 15
Safe Links not rewriting . . . 15
Safe Attachments not detonating . . . 15
Licensing gap . . . 16
Script cannot authenticate . . . 16
When to escalate to Microsoft support . . . 16
7. Best Practices — Technical Blocker Mitigation: Defender for Office 365 16
Diagnose before you touch mail flow . . . 16
Remember: one policy of each type wins per recipient . . . 17
Change one thing, on the affected recipients, then verify . . . 17
Treat routing changes as delivery-affecting . . . 17
Keep everything read-only until you deliberately fix . . . 17
Let policy changes propagate . . . 17
Keep licensing and preview facts current . . . 17
Escalate with evidence, not a description . . . 17
Where to go next . . . 17
8. Deliverables Summary — Technical Blocker Mitigation: Defender for Office 365 18
Documentation . . . 18
PowerShell scripts (Exchange Online Management / Graph advanced-hunting KQL — all read-only) 18
What the buyer receives . . . 19
Outcome . . . 19

Appendix A — PowerShell Scripts 20

Executive Summary — Technical Blocker Mitigation: Defender for Office 365

White-label · Defender

A Defender for Office 365 (MDO) rollout almost always stalls on the same short list of blockers: **mail-flow / routing conflicts** (mail bypasses filtering because it does not route through Exchange Online Protection), **policy precedence** (a higher-priority or custom policy quietly overrides the one you configured), **Safe Links / Safe Attachments not applying** to the recipients you expect, or a **licensing gap** where the plan the feature needs is not assigned. Left undiagnosed, the org thinks it is protected when it is not — or a mail-flow change breaks delivery. This solution is a **diagnostic-first playbook**: read-only discovery that pinpoints *which* blocker you actually have, the **targeted, documented manual fix**, and the verification that proves the policy now applies — so a stuck rollout gets moving again fast.

This is a **run-it-yourself** package written for an administrator with no prior deep MDO troubleshooting experience. Every script is **read-only** — it reads mail routing, policy inventory, precedence, and telemetry; it never changes a transport rule, a policy, or mail flow. Any fix is a **documented manual step** you apply deliberately, on the affected recipient subset first.

What you get

- A plain-language **Solution Overview** of the MDO signal chain — activation, mail routing through EOP, policy precedence, Safe Links / Safe Attachments scope, and verification — and where each one breaks.
- A **Requirements Guide** with verify / implement-if-missing for the diagnostic-specific prerequisites.
- An **Implementation Guide** that walks the diagnostic loop end to end — pinpoint the blocker, apply the targeted fix, re-test on the affected recipients, confirm applied, resume the rollout.
- Five **read-only PowerShell scripts** (Exchange Online Management + Microsoft Graph advanced-hunting KQL) for discovery, mail-flow / precedence analysis, blocker validation, a status report, and a support-ready data bundle.
- **Testing** and **Troubleshooting** guides that map every common symptom to its cause and fix.

What this delivers

A previously stuck Defender for Office 365 rollout **moving again** — the blocker diagnosed, the targeted fix applied and verified on the affected recipients, and, if the root cause is a Microsoft service issue rather than your configuration, a **support-ready diagnostic bundle** and clear escalation criteria.

The standard deployment model

Assess → **Configure** → **Pilot** → **Validate** → **Expand** — assess with read-only discovery to pinpoint the blocker, apply the targeted fix, pilot the fix on the affected recipient subset, validate the policy now applies, then expand by resuming the stalled rollout.

Boundary

This is a **diagnostic and troubleshooting** solution. It gets a *stalled* MDO deployment unstuck; it does **not** replace a full email-security build (see the adjacent *EOP/MDO hygiene* and *Defender for Office 365 P2 — Threat Explorer, Attack Simulation & AIR* solutions). All scripts are **read-only**. Mail-remediation and response actions (soft-delete, ZAP, tenant allow/block changes) stay **approval-gated** and are the SOC's call; incident-response and licensing determinations remain the customer's functions — this guide provides the technical diagnosis and the documented fix, not the decision.

Effort

A competent administrator can run discovery, identify the blocker, apply the documented fix, and verify that the policy applies for a single blocker class in a focused session. Tenant-wide re-scoping after the fix scales with the number of policies and recipients.

1. Solution Overview — Technical Blocker Mitigation: Defender for Office 365

What this solution is

This is a **diagnostic and troubleshooting playbook** for a **stalled Microsoft Defender for Office 365 (MDO) rollout**. It assumes you have configured MDO and hit a wall — protection that does not apply, or mail flow that fights the configuration. Its job is to answer three questions quickly and in order:

1. **Which blocker do I actually have?** (read-only discovery of routing, policy, and precedence — not guesswork)
2. **What is the exact, documented fix?** (a targeted manual step, applied deliberately)
3. **Does the policy now apply?** (verification against message trace and advanced hunting before you resume)

It targets the four blocker classes that stall almost every MDO deployment:

- **Mail-flow / routing conflicts** — mail does not route through **Exchange Online Protection (EOP)** first (misconfigured MX, an inbound connector, or a third-party gateway in front), so Safe Attachments / anti-phish never see the message.
- **Policy precedence** — a **preset security policy**, a higher-priority **custom policy**, or an **exclusion** silently overrides the policy you think is active; only one policy of each type wins per recipient.
- **Safe Links / Safe Attachments not applying** — the policy exists but is **not scoped** to the affected recipients, or is disabled/misconfigured, so links are not rewritten and attachments are not detonated.
- **Licensing gap** — the plan a feature needs (MDO P1 for Safe Links/Attachments, P2 for Threat Explorer/Attack Simulation) is not assigned. *Verify current plan requirements in your tenant.*

How Defender for Office 365 evaluation actually works (so you can find the break)

MDO has an evaluation chain, and a blocker is always a break in one specific link:

1. **Licensing/activation** — the tenant has the MDO plan and the feature is available.

2. **Mail routing** — inbound mail reaches **EOP** (correct MX / accepted domains / no bypassing connector) so the service can act on it.

3. **Policy selection & precedence** — for each recipient, MDO applies the **single highest-priority** matching policy of each type (preset > custom by priority), honoring inclusions/exclusions.

4. **Feature application** — the selected Safe Links / Safe Attachments / anti-phishing policy is **enabled** and **scoped** to the recipient, so links are rewritten and attachments detonated.

5. **Verification** — **message trace** shows the action, and advanced-hunting tables (`EmailEvents`, `UrlClickEvents`) show the protection engaging.

Find the earliest broken link and you have found the blocker. The discovery script reports the state of each link so you are not guessing.

The standard deployment model (applied to a blocker)

> ***Assess*** → ***Configure*** → ***Pilot*** → ***Validate*** → ***Expand. Assess*** *— run read-only discovery and mail-flow/precedence analysis to pinpoint the earliest broken link.* ***Configure*** *— apply the single documented fix for that blocker (a manual portal/PowerShell step, not a script action).* ***Pilot*** *— re-test on the* ***affected recipient subset*** *only.* ***Validate*** *— confirm the policy now applies via message trace and hunting.* ***Expand*** *— resume the stalled rollout to the remaining recipients.*

Scope

• Diagnosing and clearing the four blocker classes above for a stalled MDO rollout.

• Read-only discovery, mail-flow / precedence analysis, validation, status reporting, and a support-ready bundle.

• Documented manual fixes for each blocker, applied affected-recipient-first.

• Clear escalation criteria and the exact data to collect when the root cause is a Microsoft service issue.

Out of scope

• **A full email-security build / hygiene baseline** — that is the adjacent *EOP/MDO hygiene* solution; this one unblocks a rollout already in progress.

• **Deep P2 operations** (Threat Explorer hunting, Attack Simulation, AIR tuning) — see the *Defender for Office 365 P2* solution.

• **Mail remediation and response** — soft-delete, ZAP, and tenant allow/block-list changes stay **approval-gated** and are the SOC's decision.

• **Licensing determinations** — the guide tells you *which* plan a feature needs and how to verify it; assigning it is the customer's licensing function.

Connections and authentication used

• The **unified Defender portal** (`security.microsoft.com`) — **Email & collaboration** → **Policies & rules** → **Threat policies**, Explorer, and message entities.

• **Exchange admin center / Exchange Online Management PowerShell** — accepted domains, connectors, preset and custom threat policies, and message trace (all read-only here).

• **Microsoft Graph advanced-hunting API** (via `Microsoft.Graph`) for read-only `EmailEvents` / `UrlClickEvents` validation.

• Least-privilege roles: **Security Reader** / **Global Reader** / **View-Only** Exchange roles for all read-only diagnostics; **Security Administrator** (and the appropriate Exchange role) only to apply a fix.

2. Requirements Guide — Technical Blocker Mitigation: Defender for Office 365

Verify **every** requirement before diagnosing. Each entry: **Description · Why · Verify · Implement-if-missing.** Category-wide prerequisites (tenant, Defender licensing, admin roles, onboarding reach, network access, pilot ring, workspace, response governance) are in `global-requirements-defender.md` — do not repeat them; the items below are **specific to running the MDO blocker diagnostics**. Objects you create are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

2.1 Read-only access to Exchange Online and the Defender portal

Description: A read-only footprint: **Global Reader** or **Security Reader** in the Defender portal, plus a **View-Only** Exchange role (View-Only Configuration / View-Only Recipients) and consent to the read-only Microsoft Graph advanced-hunting scope (`ThreatHunting.Read.All`). **Why:** The diagnostics read policy inventory, precedence, mail routing, message trace, and `EmailEvents` — all read-only; no write role is needed to diagnose. **Verify:** Connect with `Connect-ExchangeOnline` and run `Get-SafeLinksPolicy` (read) without error; confirm `security.microsoft.com` → **Email & collaboration** loads. `TBMMDO-Discovery.ps1` reports the roles it resolved. **Implement-if-missing:** Assign Global/Security Reader and a View-Only Exchange role; grant admin consent to `ThreatHunting.Read.All`.

2.2 Exchange Online Management module available

Description: The **ExchangeOnlineManagement** PowerShell module (v3+) on the workstation running the diagnostics. **Why:** Accepted domains, connectors, preset/custom threat policies, precedence, and message trace are exposed through Exchange Online cmdlets, not Graph. **Verify:** `Get-Module -ListAvailable ExchangeOnlineManagement` returns v3 or later; `Connect-ExchangeOnline` succeeds. `TBMMDO-Discovery.ps1` installs it if missing. **Implement-if-missing:** `Install-Module ExchangeOnlineManagement -Scope CurrentUser`.

2.3 The affected recipient subset defined

Description: A named subset of the mailboxes/recipients exhibiting the blocker — the ones you will re-test the fix against before resuming. Name it `MDO Blocker Triage – Affected Recipients – Diagnose`. **Why:** Fixes are validated on the affected recipients first (per `global-requirements-defender.md` G6) so you prove the policy applies before touching the tenant. **Verify:** A mail-enabled group / list of the affected recipients exists; `TBMMDO-Validate.ps1` checks the

policy that applies to them. **Implement-if-missing:** Create the group `MDO Blocker Triage – Affected Recipients – Diagnose` and add the affected mailboxes.

2.4 Knowledge of the mail-routing topology

Description: Know how inbound mail reaches the tenant — direct **MX** → **EOP**, or via a **third-party gateway / inbound connector / hybrid** in front of EOP. **Why:** Routing conflicts are a top blocker; if a gateway strips or pre-scans mail, MDO features never engage. You must know the real path to diagnose it. **Verify:** `Get-AcceptedDomain`, `Get-InboundConnector`, and the public MX record for the domain; `TBMMDO-MailFlow.ps1` reports accepted domains, connectors, and routing hints. **Implement-if-missing:** Confirm the routing design with whoever owns mail flow; document any gateway in front of EOP.

2.5 A safe test message method

Description: A benign way to exercise Safe Links / Safe Attachments — a test URL and the standard Microsoft test procedure (e.g. an EICAR-style attachment through the documented Microsoft test), sent to a recipient in the affected subset. **Why:** Verification needs a controlled message that *should* trigger the feature so you can confirm rewriting/detonation and see it in message trace and hunting. **Verify:** You can send a controlled test to `MDO Blocker Triage – Affected Recipients – Diagnose` and view the result in message trace. **Implement-if-missing:** Prepare the benign test material per current Microsoft guidance. *Verify the current recommended test procedure.*

2.6 A place to store the diagnostic output and support bundle

Description: A secure location for the JSON snapshots the scripts write and for the support bundle `MDO Support Bundle – <case> – Escalate`. **Why:** The before/after snapshots prove the policy now applies, and the bundle is what you attach to a Microsoft support case. **Verify:** A working folder exists with least-privilege access; `TBMMDO-Report.ps1` writes there. **Implement-if-missing:** Create a restricted working folder; do not store recipient identifiers in shared/unsecured locations.

Checklist

- Read-only Defender + View-Only Exchange roles + `ThreatHunting.Read.All` — `TBMMDO-Discovery.ps1`
- ExchangeOnlineManagement module (v3+) installed
- Affected recipient subset `MDO Blocker Triage – Affected Recipients – Diagnose` created
- Mail-routing topology known (MX / connectors / gateway) — `TBMMDO-MailFlow.ps1`
- Safe test-message method prepared
- Secure working folder for snapshots and the support bundle

Global Requirements — Defender (category-level)

These requirements apply to every solution in the Defender category (Microsoft Defender XDR and Defender for Cloud). Published once at the category level so listings stay small. Confirm before deploying any Defender solution. Each entry: Description / Why / Verify / Implement-if-missing.

> *Defender changes touch detection and response for the whole tenant. Onboard in a ring (pilot devices/users first), keep automated response in an approval/simulate posture until trusted, and never disable an existing control to "test."*

G1. Microsoft 365 / Entra tenant

Description: An active Microsoft 365 tenant with Entra ID. Why: Defender XDR services and the unified portal (security.microsoft.com) are tenant workloads. Verify: Sign in to `https://security.microsoft.com` as an administrator. Implement: Provision a Microsoft 365 tenant and confirm Entra ID identities exist.

G2. Defender licensing

Description: The Defender plans for the pillars in scope — e.g. Defender for Endpoint P1/P2, Defender for Identity, Defender for Office 365 P1/P2, Defender for Cloud Apps, and Defender for Cloud plans. Why: Each pillar/feature is license-gated; onboarding without the plan fails or silently limits features. Verify: Admin center -> `Billing -> Licenses` (or the Defender portal). *Confirm the exact plan each feature needs — verify current availability in your tenant.* Implement: Assign the required plans (often bundled in M365 E5 / E5 Security) before onboarding.

G3. Administrative roles (least privilege)

Description: Security Administrator (and Global Administrator for first-time service activation). Why: Onboarding, RBAC, and policy changes in the Defender portal require these roles. Verify: Entra -> `Roles and administrators`; confirm Security Administrator. Implement: Assign Security Administrator; reserve Global Administrator for one-time activations.

G4. Onboarding reach to endpoints / identity / workloads

Description: A management channel to onboard the pillars — Intune/Configuration Manager/GPO/script for endpoints, domain controllers/AD FS for Identity sensors, and cloud connectors for Defender for Cloud. Why: Each pillar collects signal from a source that must be reachable and instrumented. Verify: Endpoints are managed (Intune/CM); DCs are reachable; cloud subscriptions are connected. Run the solution's discovery script. Implement: Establish the onboarding channel per pillar (see the relevant catalog endpoint/identity solutions).

G5. Network access to Defender service endpoints

Description: Outbound HTTPS to the Microsoft Defender / Microsoft 365 service URLs from onboarded assets. Why: Sensors and agents report to the cloud service; blocked endpoints break onboarding and detections. Verify: Confirm the documented Defender endpoint URLs/IP ranges are allowed. *Verify current endpoint list.* Implement: Allow the Defender/M365 endpoints through proxy/firewall.

G6. A pilot ring

Description: A defined set of pilot devices/users/subscriptions to onboard first. Why: XDR onboarding and automated response are validated on a ring before tenant-wide rollout. Verify: A pilot group exists in Entra/Intune. Implement: Create the pilot ring before enabling anything that acts automatically.

G7. Log Analytics workspace (where required)

Description: A Log Analytics workspace for Defender for Cloud / advanced hunting export where used. Why: Some Defender data and continuous export land in a workspace. Verify: A workspace exists and is access-controlled. Run the discovery script. Implement: Provision a workspace (or reuse the SIEM/observability one) with least-privilege access.

G8. Response governance

Description: An agreed position on automated response (auto-remediation levels, approval requirements). Why: XDR/AIR can take disruptive actions (isolate, delete, disable); the organization must decide the guardrails. Verify: A documented response policy exists; automation is in an approval/simulate posture until trusted. Implement: Define response guardrails; keep AIR/automation approval-gated at first. *Incident-response decisions are the SOC/security team's; this guide covers the technical configuration.*

The deployment model (all solutions): Assess -> Configure -> Pilot -> Validate -> Expand.

3. Implementation Guide — Technical Blocker Mitigation: Defender for Office 365

Follow these steps **in order**. This is a **diagnostic loop**, not a build: you assess to pinpoint the blocker, apply one targeted documented fix, re-test the affected recipients, and only then resume the rollout. All scripts are **read-only**; every *fix* is a manual step you apply deliberately. Objects are named `<Purpose> – <Scope> – <Stage>` and reused verbatim in every later step, script, and test.

> ***Model:*** *Assess (pinpoint) → Configure (fix) → Pilot (re-test affected recipients) → Validate (confirm applied) → Expand (resume rollout). Never change mail flow to "test," and keep mail-remediation actions approval-gated throughout.*

Naming (set once, reuse verbatim)

- Affected recipient subset: `MDO Blocker Triage – Affected Recipients – Diagnose`
- Advanced-hunting saved query (policy/telemetry check): `MDO Policy Blocker Check – Tenant – Diagnose`
- Support bundle output: `MDO Support Bundle – <case> – Escalate`
- Discovery snapshot file: `MDO-blocker-discovery.json`

Step 1 — Assess: pinpoint the blocker (read-only)

1. Connect read-only: `Connect-ExchangeOnline` (View-Only role) and, for hunting, `Connect-MgGraph`.

2. Run `TBMMDO-Discovery.ps1`. It captures the evaluation chain in one snapshot (`MDO-blocker-discovery.json`): licensed MDO plans, accepted domains, preset-security-policy state, custom Safe Links / Safe Attachments / anti-phishing policies with their **priority order**, whether each is **enabled**, and their **scope** (included/excluded recipients).

3. Assemble `MDO Blocker Triage – Affected Recipients – Diagnose` from the mailboxes showing the symptom.

4. Read the snapshot top-down against the evaluation chain (Overview §"How evaluation actually works") and identify the **earliest broken link** — plan, routing, precedence, or feature scope.

Step 2 — Assess: analyze mail routing and precedence (read-only)

1. Run `TBMMDO-MailFlow.ps1`. It reports accepted domains, inbound connectors, and any gateway/hybrid routing hints — confirming whether inbound mail actually reaches EOP first.

2. Review the **policy precedence** section of the discovery output: for the affected recipients, which **single** policy of each type actually wins (preset over custom, by priority). A "configured but never-applied" policy is almost always losing on precedence or scope.

3. Record which link is broken before changing anything.

Step 3 — Configure: apply the one targeted fix (manual, documented)

Apply **only** the fix for the blocker you identified. Each is a deliberate manual step — the scripts never make these changes.

• **Mail-flow / routing conflict:** Correct the routing so inbound mail reaches **EOP** first — fix the public **MX**, or adjust/remove the **inbound connector** or third-party gateway that bypasses filtering. *This changes mail delivery — pilot on the affected recipients' domain path and confirm delivery before wider change.*

• **Policy precedence:** In **Email & collaboration** → **Policies & rules** → **Threat policies**, adjust **priority** so the intended policy wins for the affected recipients, or remove the **exclusion** that drops them out of scope. Remember only one policy of each type applies per recipient.

• **Safe Links / Safe Attachments not applying:** Enable the policy and add `MDO Blocker Triage – Affected Recipients – Diagnose` to its **included** scope (or use a preset policy scoped to them). Confirm the feature toggles (URL rewrite / dynamic delivery) are on.

• **Licensing gap:** Assign the MDO plan the feature needs (P1 for Safe Links/Attachments, P2 for Explorer/Attack Sim) in **Billing** → **Licenses**. *Verify the exact plan requirement in your tenant.*

Step 4 — Pilot: re-test the fix on the affected recipients only

1. Apply the fix scoped to `MDO Blocker Triage – Affected Recipients – Diagnose` only — not the tenant.

2. Send the safe test message (Requirements 2.5) to a recipient in the subset.

3. Give policy propagation time (threat-policy changes can take time to apply), then validate.

Step 5 — Validate: confirm the policy now applies

1. Run `TBMMDO-Validate.ps1`. It runs the saved query `MDO Policy Blocker Check – Tenant – Diagnose` against `EmailEvents` / `UrlClickEvents` for the affected recipients and confirms the protection engaged (link rewritten / attachment detonated / message acted on).

2. Cross-check in **message trace** that the expected MDO action appears for the test message.

3. Run `TBMMDO-Report.ps1` for the before/after evidence.

4. Complete the **Testing Guide** for the blocker class before expanding.

Step 6 — Expand: resume the stalled rollout

1. With the affected recipients green, broaden the corrected policy scope / preset to the remaining recipients in waves.

2. Re-run `TBMMDO-Validate.ps1` after each wave and compare with `TBMMDO-Report.ps1`.

3. If a wave re-introduces the symptom, return to Step 1 for that subset — precedence can differ by recipient group.

Step 7 — Escalate (only if it is a service issue, not your config)

1. If routing reaches EOP, the winning policy is correct and scoped, the plan is assigned, and the test still is not protected, it may be a Microsoft service issue.

2. Run `TBMMDO-SupportBundle.ps1` to assemble `MDO Support Bundle – <case> – Escalate` — the discovery snapshot, mail-flow analysis, the affected recipients, message-trace IDs, and timestamps.

3. Open a Microsoft support case, attach the bundle, and reference the escalation criteria in the Troubleshooting Guide. *Incident-response decisions remain the SOC's; this is a service-support escalation.*

Order of operations

`Assess (discovery + mail-flow/precedence) → identify earliest broken link → apply the one targeted fix → re-test the affected recipients → validate the policy applies → resume rollout → escalate with a bundle only if it is a service issue`

5. Testing Guide — Technical Blocker Mitigation: Defender for Office 365

Run these checks after applying a fix on the affected recipients (`MDO Blocker Triage – Affected Recipients – Diagnose`) and again after each expansion wave. Each test has a **pass/fail gate**; do not resume the wider rollout until the affected recipients pass the test for their blocker class. Objects are referenced by the exact names from the Implementation Guide.

Test 1 — Blocker pinpointed (baseline captured)

Procedure: Run `TBMMDO-Discovery.ps1` and confirm `MDO-blocker-discovery.json` has the plan, policy inventory, priority order, enable-state, and scope populated. **Pass:** A single earliest broken link is identifiable; the affected recipients are in the triage group. **Fail:** Snapshot empty or access-denied — fix read access (Requirements 2.1) and re-run.

Test 2 — Mail routing reaches EOP

Procedure: Run `TBMMDO-MailFlow.ps1`. Confirm the public MX resolves to EOP (or a documented gateway that forwards to EOP), accepted domains are correct, and no inbound connector bypasses filtering for the affected recipients. **Pass:** Inbound mail for the affected recipients reaches EOP first. **Fail:** Mail bypasses EOP — see Troubleshooting *Mail bypasses filtering (routing conflict)*.

Test 3 — Correct policy wins on precedence

Procedure: Run `TBMMDO-Discovery.ps1`; in the precedence section confirm the **single** highest-priority policy of each type that applies to the affected recipients is the intended one. **Pass:** The intended Safe Links / Safe Attachments / anti-phish policy is the winner for the affected recipients. **Fail:** A preset or higher-priority custom policy or an exclusion wins instead — see Troubleshooting *Policy configured but never applies (precedence)*.

Test 4 — Safe Links rewriting

Procedure: Send the safe test URL to a recipient in the subset; run `TBMMDO-Validate.ps1` and check `UrlClickEvents` / message trace. **Pass:** The URL is rewritten/wrapped and the click is recorded by MDO. **Fail:** URL not rewritten — see Troubleshooting *Safe Links not rewriting*.

Test 5 — Safe Attachments detonation

Procedure: Send the benign test attachment (standard Microsoft procedure) to a recipient in the subset; run `TBMMDO-Validate.ps1` and check `EmailEvents` / message trace for the Safe Attachments action. **Pass:** The attachment is detonated/held per policy and the action appears. **Fail:** No Safe Attachments action — see Troubleshooting *Safe Attachments not detonating*.

Test 6 — Licensing present

Procedure: Run `TBMMDO-Discovery.ps1` and confirm the MDO plan the feature needs is reported as licensed. **Pass:** The required MDO plan (P1/P2) is present. *Verify current plan requirements in your tenant.* **Fail:** Plan missing — see Troubleshooting *Licensing gap*.

Test 7 — Before/after evidence produced

Procedure: Run `TBMMDO-Report.ps1` and confirm it emits a before/after comparison from the Step 1 snapshot versus current state. **Pass:** The report shows the blocker's link moved from broken to cleared. **Fail:** No change captured — re-run discovery to refresh the baseline.

Test 8 — Escalation bundle valid (only if escalating)

Procedure: If protection still does not apply after a correct fix, run `TBMMDO-SupportBundle.ps1` and confirm `MDO Support Bundle – <case> – Escalate` contains the discovery snapshot, mail-flow analysis, affected recipients, message-trace IDs, and timestamps. **Pass:** A complete, attachable bundle exists. **Fail:** Bundle incomplete — re-run discovery/mail-flow first, then rebuild.

Exit gate

The affected recipients pass the tests for their blocker class **before** you resume the wider rollout (Implementation Step 6). Re-run Tests 3–5 and 7 after each expansion wave and compare against the saved baseline.

6. Troubleshooting Guide — Technical Blocker Mitigation: Defender for Office 365

The four blocker classes in **symptom** → **cause** → **fix** form, plus escalation. Run `TBMMDO-Discovery.ps1` and `TBMMDO-MailFlow.ps1` first — most causes surface there. This *is* the troubleshooting solution, so this guide is the core reference.

Mail bypasses filtering (routing conflict)

Symptom: Messages arrive without any MDO stamps; Safe Attachments / anti-phish never act; `EmailEvents` shows mail not scanned by EOP. **Cause:** Inbound mail does not reach **EOP** first — MX points elsewhere, or an inbound connector / third-party gateway / hybrid path pre-delivers or forwards mail past filtering. **Fix:** Run `TBMMDO-MailFlow.ps1`; correct the public **MX** to EOP, or adjust/remove the **inbound connector** or gateway that bypasses filtering (per Requirements 2.4). *This changes delivery — pilot on the affected recipients' path and confirm mail flows before wider change.* Re-run `TBMMDO-Validate.ps1`.

Policy configured but never applies (precedence)

Symptom: A Safe Links / Safe Attachments / anti-phish policy exists and looks correct, but the affected recipients get a different (or no) protection. **Cause:** A **preset security policy** or a higher-priority **custom policy** wins, or the affected recipients are in the policy's **exclusion** list — only one policy of each type applies per recipient. **Fix:** In `TBMMDO-Discovery.ps1` output, find the **winning** policy for the affected recipients. Adjust **priority** so the intended policy wins, or remove the exclusion, in **Threat policies**. Do not clone more policies. Re-run `TBMMDO-Validate.ps1`.

Safe Links not rewriting

Symptom: URLs in test mail are not wrapped/rewritten; `UrlClickEvents` shows nothing for the affected recipients. **Cause:** The winning Safe Links policy is disabled, not scoped to the recipients, or URL-rewrite is off; or mail bypassed EOP (see routing). **Fix:** Confirm routing (Test 2). Enable the Safe Links policy and add `MDO Blocker Triage – Affected Recipients – Diagnose` to its included scope; confirm URL rewrite is on. Allow propagation, send the safe test URL, re-run `TBMMDO-Validate.ps1`.

Safe Attachments not detonating

Symptom: Test attachment is delivered without detonation/hold; no Safe Attachments action in message trace. **Cause:** The winning Safe Attachments policy is disabled or unscoped, the action is set to Off/Monitor, or mail bypassed EOP. **Fix:** Confirm routing. Enable the policy, scope it to the affected recipients, set the intended action (Block/Dynamic Delivery per your design). Allow propagation, send the benign test attachment (standard Microsoft procedure), re-run `TBMMDO-Validate.ps1`.

Licensing gap

Symptom: Safe Links/Attachments (P1) or Explorer/Attack Simulation (P2) options are missing or greyed out. **Cause:** The MDO plan the feature needs is not assigned. **Fix:** Assign the plan (**Billing** → **Licenses**). *Verify the exact plan the feature needs — availability changes; confirm in your tenant.* Re-run `TBMMDO-Discovery.ps1`.

Script cannot authenticate

Symptom: `TBMMDO-*.ps1` fails on `Connect-ExchangeOnline` / `Connect-MgGraph` or returns access-denied. **Cause:** The ExchangeOnlineManagement module is missing, the account lacks a View-Only Exchange / Reader role, or `ThreatHunting.Read.All` consent was not granted. **Fix:** `Install-Module ExchangeOnlineManagement`; sign in with a Global/Security Reader + View-Only Exchange role; consent to `ThreatHunting.Read.All`. Re-run.

When to escalate to Microsoft support

Escalate — and attach `MDO Support Bundle – <case> – Escalate` — when **all** of the following hold:

- Mail routing reaches EOP (Test 2 passes).
- The **winning** policy for the affected recipients is the intended one, enabled and scoped (Tests 3–5).
- The required MDO plan is assigned (Test 6).
- Policy propagation time elapsed, yet the test message still is not protected.

In that case the root cause is likely a **service-side issue**, not your configuration. Run `TBMMDO-SupportBundle.ps1`, open the case, and attach the bundle with message-trace IDs. *Incident-response decisions remain the SOC's; this is a service-support escalation.*

7. Best Practices — Technical Blocker Mitigation: Defender for Office 365

Field-proven guidance for diagnosing a stalled MDO rollout cleanly, fixing the *right* thing, and never breaking mail delivery in the process.

Diagnose before you touch mail flow

- Run `TBMMDO-Discovery.ps1` and `TBMMDO-MailFlow.ps1` **first**. Read the evaluation chain top-down; the earliest broken link is the blocker. Changing a transport rule or connector on a hunch can break delivery for the whole org.

• A "policy that never applies" is almost always a **precedence or scope** problem, not a broken feature — confirm which policy actually wins before editing the feature.

Remember: one policy of each type wins per recipient

• MDO applies the **single highest-priority** matching policy of each type (preset over custom, by priority). If your policy is not winning, fix **priority or scope**, do not clone more policies — that compounds the precedence problem.

Change one thing, on the affected recipients, then verify

• Apply exactly **one** documented fix, scoped to `MDO Blocker Triage – Affected Recipients – Diagnose`, then run `TBMMDO-Validate.ps1`. One change at a time keeps cause and effect clear.

• Never resume the tenant-wide rollout until the affected recipients pass their test gate.

Treat routing changes as delivery-affecting

• MX, connector, and gateway changes affect **mail delivery**, not just security. Pilot on the affected recipients' path, confirm mail still flows, and have a rollback ready before any wider change.

Keep everything read-only until you deliberately fix

• Every script here is read-only. Treat the *fix* as a separate, conscious action in the portal/Exchange PowerShell. Keep mail-remediation actions (soft-delete, ZAP, allow/block-list edits) **approval-gated** — those are the SOC's call.

Let policy changes propagate

• Threat-policy edits and scope changes take time to apply across the service. Give a fix propagation time before re-testing, then validate with a fresh test message.

Keep licensing and preview facts current

• MDO features are **plan-gated** (P1 for Safe Links/Attachments, P2 for Explorer/Attack Simulation). Before concluding "it's broken," confirm the feature is even licensed — *verify current availability and plan requirements in your tenant.*

Escalate with evidence, not a description

• If protection survives a correct fix, attach `MDO Support Bundle – <case> – Escalate` with message-trace IDs to the case. A complete bundle resolves cases far faster than a written symptom.

Where to go next

Once the rollout is unblocked, natural next steps in the catalog:

• **EOP / MDO email-security hygiene** to bring the whole email-protection baseline to standard.

• **Defender for Office 365 (P2) — Threat Explorer, Attack Simulation & AIR** to operate the email pillar deeply.

• **Defender XDR Deployment & Unified Onboarding** to correlate email alerts with endpoint, identity, and cloud-app signal in one portal.

• The **Sentinel** SOC/XDR integration to stream MDO signal into a SIEM for long-term hunting.

8. Deliverables Summary — Technical Blocker Mitigation: Defender for Office 365

Documentation

Document	Purpose
Executive Summary	The blockers, the diagnostic-first approach, the outcome, the boundary, effort
1. Solution Overview	The four MDO blocker classes, the evaluation chain (routing → precedence → feature scope), where each breaks
2. Requirements Guide	Diagnostic-specific prerequisites with Verify / Implement-if-missing
Global Requirements (Defender)	Category-wide prerequisites, published once
3. Implementation Guide	The diagnostic loop — assess, fix, re-test the affected recipients, validate, resume, escalate
5. Testing Guide	Eight pass/fail tests, one per link in the evaluation chain
6. Troubleshooting Guide	Every blocker class in symptom → cause → fix, plus escalation criteria
7. Best Practices	Diagnose-first discipline, precedence rules, and where to go next

PowerShell scripts (Exchange Online Management / Graph advanced-hunting KQL — all read-only)

Script	What it does
`TBMMDO-Discovery.ps1`	Read-only evaluation-chain snapshot — licensed MDO plans, accepted domains, preset & custom Safe Links / Safe Attachments / anti-phish policies with priority order, enable-state, and recipient scope
`TBMMDO-MailFlow.ps1`	Read-only mail-routing analysis — accepted domains, inbound connectors, gateway/hybrid hints — confirms whether inbound mail reaches EOP before MDO can act

Script	What it does
`TBMMDO-Validate.ps1`	Read-only pass/fail for the affected recipients — runs the saved hunting query against `EmailEvents` / `UrlClickEvents` to confirm Safe Links rewriting and Safe Attachments action
`TBMMDO-Report.ps1`	Read-only before/after status report from the discovery baseline versus current state — the evidence the policy now applies
`TBMMDO-SupportBundle.ps1`	Read-only assembly of the support-ready bundle (discovery snapshot, mail-flow analysis, affected recipients, message-trace IDs, timestamps) for a Microsoft support case

What the buyer receives

- This solution as a **Word** document and **PDF**.
- All five **read-only PowerShell scripts** (with embedded KQL) in a single ZIP.
- A repeatable diagnostic process following the model **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — pinpoint the blocker, apply one documented fix, re-test the affected recipients, confirm the policy applies, resume the rollout.

Delivered as a **Complete Solution Package (PDF + Word + scripts)**.

Outcome

A previously stuck Defender for Office 365 rollout moving again — the blocker diagnosed against the evaluation chain, the targeted documented fix applied and verified on the affected recipients, and a support-ready bundle ready if the root cause turns out to be a Microsoft service issue.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

TBMMDO-Discovery.ps1

Purpose: Read-only discovery for the "Technical Blocker Mitigation — Defender for Office 365" solution. Captures the MDO evaluation chain in one snapshot so you can find the earliest broken link: licensed MDO plans, accepted domains, preset-security-policy state, and the custom Safe Links / Safe Attachments / anti-phishing policies with their PRIORITY order, ENABLE state, and recipient SCOPE (inclusions/exclusions). Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMDO-Discovery.ps1 Requires the ExchangeOnlineManagement module (v3+) and a View-Only Exchange / Reader role. Connects with Connect-ExchangeOnline (read-only cmdlets only). Graph is used only to read licensed plans. EXPECTED RESULTS A JSON snapshot (MDO-blocker-discovery.json) plus a console summary. Read it top-down against the evaluation chain in the Solution Overview; the earliest broken link is the blocker. .NOTES White-label. Read-only — uses Get-* cmdlets only; changes no policy or mail flow. Licensing/plan facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDO-blocker-discovery.json")

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMDO-Discovery.ps1 Requires the ExchangeOnlineManagement module (v3+) and a View-Only Exchange / Reader role. Connects with Connect-ExchangeOnline (read-only cmdlets only). Graph is used only to read licensed plans. EXPECTED RESULTS A JSON snapshot (MDO-blocker-discovery.json) plus a console summary. Read it top-down against the evaluation chain in the Solution Overview; the earliest broken link is the blocker. .NOTES White-label. Read-only — uses Get-* cmdlets only; changes no policy or mail flow. Licensing/plan facts are indicative — verify current availability in your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDO-blocker-discovery.json")

TBMMDO-MailFlow.ps1

Purpose: The top MDO blocker is inbound mail that does not reach Exchange Online Protection (EOP) first, so Safe Attachments / anti-phish never act. This script reports accepted domains, inbound connectors, and public MX resolution for each domain, and flags routing that could bypass filtering (a gateway in front of EOP, an inbound connector, or an MX that does not point to EOP). Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMDO-MailFlow.ps1 Requires the ExchangeOnlineManagement module (v3+) and a View-Only Exchange / Reader role. EXPECTED RESULTS A per-domain routing verdict (reaches EOP / possible bypass) and a connector inventory, written to MDO-mailflow.json. A possible-bypass verdict points the fix at routing, NOT at MDO policy. .NOTES White-label. Read-only — Get-* and DNS lookups only; changes no mail flow. MX/EOP host patterns evolve — verify the current EOP MX target for your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDO-mailflow.json")

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMDO-MailFlow.ps1 Requires the ExchangeOnlineManagement module (v3+) and a View-Only Exchange / Reader role. EXPECTED RESULTS A per-domain routing verdict (reaches EOP / possible bypass) and a connector inventory, written to MDO-mailflow.json. A possible-bypass verdict points the fix at routing, NOT at MDO policy. .NOTES White-label. Read-only — Get-* and DNS lookups only; changes no mail flow. MX/EOP host patterns evolve — verify the current EOP MX target for your tenant. > [CmdletBinding()] param([string]$OutputPath = ".\MDO-mailflow.json")

TBMMDO-Report.ps1

Purpose: Read-only reporting for the "Technical Blocker Mitigation — Defender for Office 365" solution. Compares the discovery baseline (MDO-blocker-discovery.json) with a fresh read of the same evaluation-chain links (plan, preset/custom policy state and priority) and prints a before/after summary — the evidence the blocker moved from broken to cleared. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMDO-Report.ps1 -Baseline .\MDO-blocker-discovery.json Requires the ExchangeOnlineManagement module (v3+) and a View-Only Exchange / Reader role. EXPECTED RESULTS A table of each link's baseline vs current state and an overall CLEARED / STILL-BLOCKED verdict, written to MDO-blocker-report.json. .NOTES White-label. Read-only. Reads state only; changes nothing. > [CmdletBinding()] param([string]$Baseline = ".\MDO-blocker-discovery.json", [string]$OutputPath = ".\MDO-blocker-report.json")

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMDO-Report.ps1 -Baseline .\MDO-blocker-discovery.json Requires the ExchangeOnlineManagement module (v3+) and a View-Only Exchange / Reader role. EXPECTED RESULTS A table of each link's baseline vs current state and an overall CLEARED / STILL-BLOCKED verdict, written to MDO-blocker-report.json. .NOTES White-label. Read-only. Reads state only; changes nothing. > [CmdletBinding()] param([string]$Baseline = ".\MDO-blocker-discovery.json", [string]$OutputPath = ".\MDO-blocker-report.json")

TBMMDO-SupportBundle.ps1

Purpose: Read-only packaging for the "Technical Blocker Mitigation — Defender for Office 365" solution. When routing reaches EOP, the winning policy is correct and scoped, the plan is assigned, and protection still does not apply (likely a Microsoft service issue), this assembles the bundle 'MDO Support Bundle - <case> - Escalate': the discovery snapshot, the mail-flow analysis, the affected recipients, message-trace IDs, tenant context, and timestamps. Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMDO-SupportBundle.ps1 -Case "MSCASE-12345" -MessageIds "id1,id2" Point it at the artifacts you already produced: .\TBMMDO-SupportBundle.ps1 -Case "MSCASE-12345" -Discovery .\MDO-blocker-discovery.json -MailFlow .\MDO-mailflow.json EXPECTED RESULTS A single folder 'MDO-Support-Bundle-<case>' with a manifest and copies of the read-only artifacts, ready to attach to a Microsoft support case. .NOTES White-label. Read-only — collects existing read-only artifacts; changes nothing. Do not place the bundle in shared/unsecured storage; it contains recipient/tenant identifiers. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Case, [string]$MessageIds = "", [string]$Discovery = ".\MDO-blocker-discovery.json", [string]$MailFlow = ".\MDO-mailflow.json", [string]$Report = ".\MDO-blocker-report.json")

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMDO-SupportBundle.ps1 -Case "MSCASE-12345" -MessageIds "id1,id2" Point it at the artifacts you already produced: .\TBMMDO-SupportBundle.ps1 -Case "MSCASE-12345" -Discovery .\MDO-blocker-discovery.json -MailFlow .\MDO-mailflow.json EXPECTED RESULTS A single folder 'MDO-Support-Bundle-<case>' with a manifest and copies of the read-only artifacts, ready to attach to a Microsoft support case. .NOTES White-label. Read-only — collects existing read-only artifacts; changes nothing. Do not place the bundle in shared/unsecured storage; it contains recipient/tenant identifiers. > [CmdletBinding()] param([Parameter(Mandatory)][string]$Case, [string]$MessageIds = "", [string]$Discovery = ".\MDO-blocker-discovery.json", [string]$MailFlow = ".\MDO-mailflow.json", [string]$Report = ".\MDO-blocker-report.json")

TBMMDO-Validate.ps1

Purpose: Read-only validation for the "Technical Blocker Mitigation — Defender for Office 365" solution. Runs the saved advanced-hunting logic 'MDO Policy Blocker Check - Tenant - Diagnose' via Microsoft Graph Security to confirm, after a fix and a test message, that MDO protection now engages for the affected recipients: Safe Links rewriting (UrlClickEvents) and Safe Attachments / threat actions (EmailEvents). Makes NO changes. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\TBMMDO-Validate.ps1 -RecipientLike "affected.user" Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS PASS/FAIL lines for Safe Links and Safe Attachments engagement. FAIL points you to the matching Troubleshooting section for that blocker class. .NOTES White-label. Read-only. Advanced-hunting queries only read telemetry; they change nothing. > [CmdletBinding()] param([string]$RecipientLike = "", [int]$LookbackHours = 24)

How to run: powershell -ExecutionPolicy Bypass -File .\TBMMDO-Validate.ps1 -RecipientLike "affected.user" Requires Microsoft.Graph.Authentication and ThreatHunting.Read.All consent. EXPECTED RESULTS PASS/FAIL lines for Safe Links and Safe Attachments engagement. FAIL points you to the matching Troubleshooting section for that blocker class. .NOTES White-label. Read-only. Advanced-hunting queries only read telemetry; they change nothing. > [CmdletBinding()] param([string]$RecipientLike = "", [int]$LookbackHours = 24)